

Deep Learning

Lecture 3: Designing Architectures

Amir Atapour-Abarghouei

amir.atapour-abarghouei@durham.ac.uk

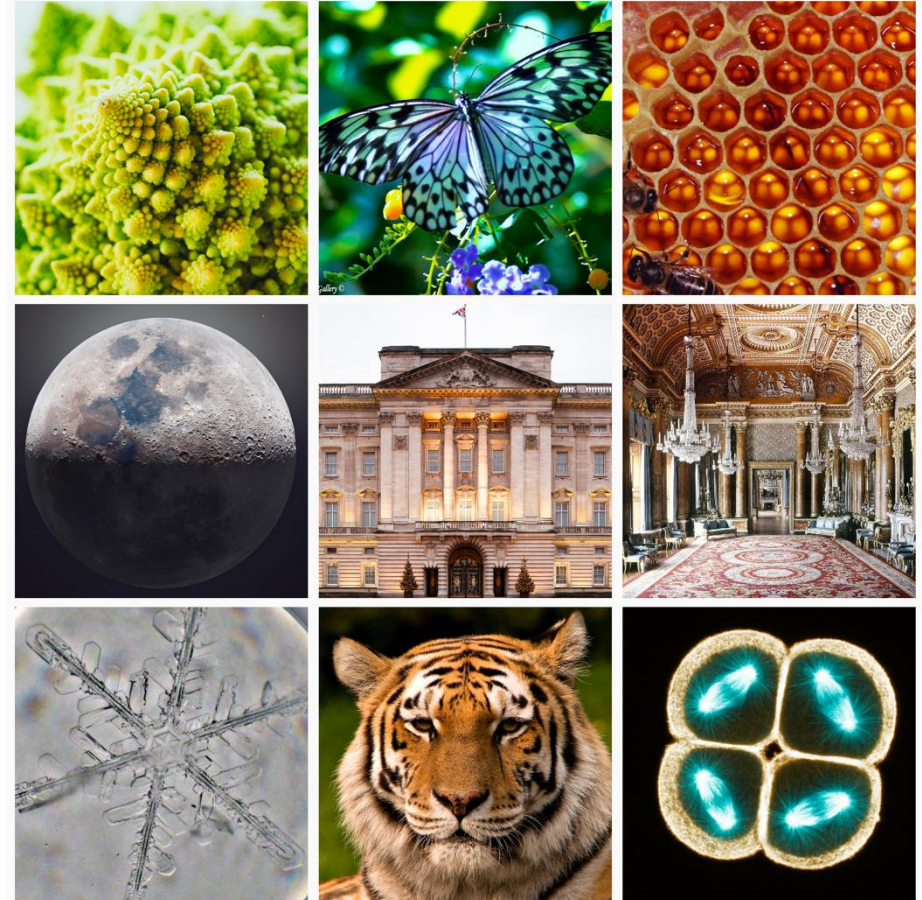
Durham University



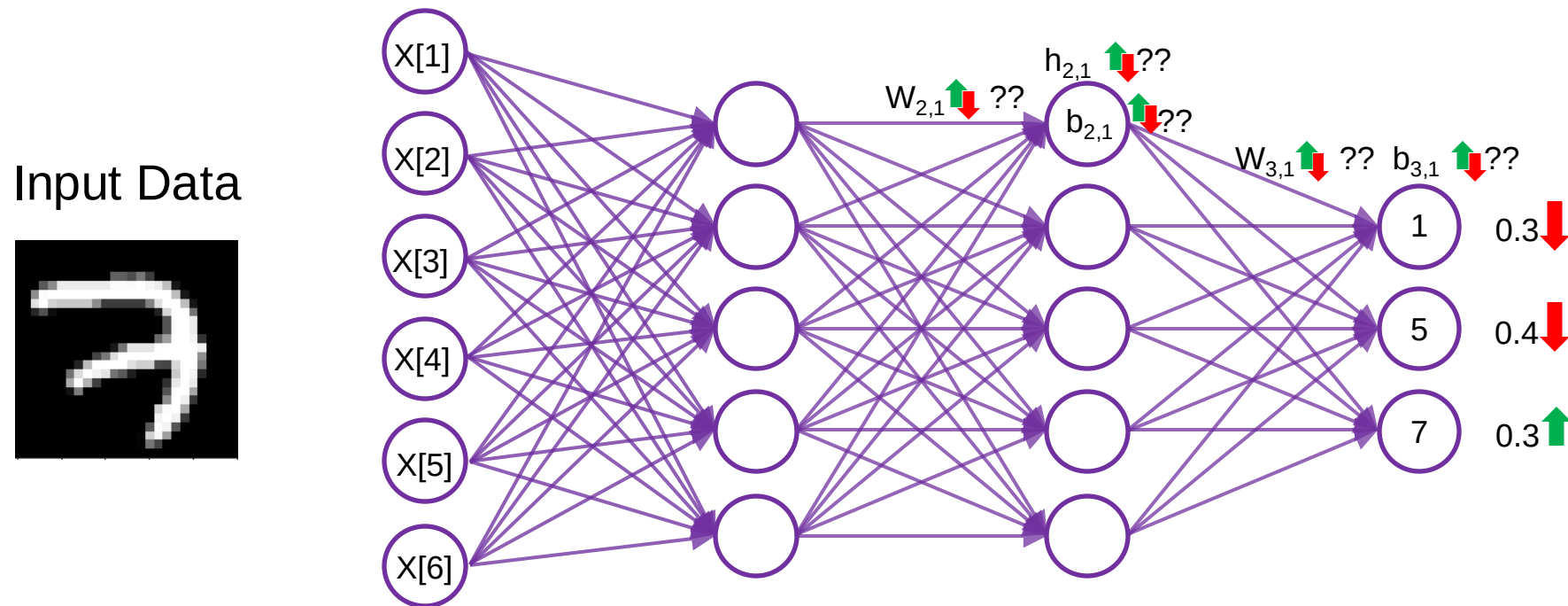
Lecture Overview



- 1 Training a Neural Network
- 2 Convolutional Neural Networks
- 3 Transpose Convolutions



The only things we can change are the weights and bias's



- Backpropagation
- Normally done on batches of samples



- Let's take a (very) simple Neural Network
- Only one neuron per layer
- Each layer is applying a function to the output of the previous layer
- Let's say that:

$$t = f(x) = \sigma(x * w_1 + b_1)$$

$$u = g(t) = \sigma(t * w_2 + b_2)$$

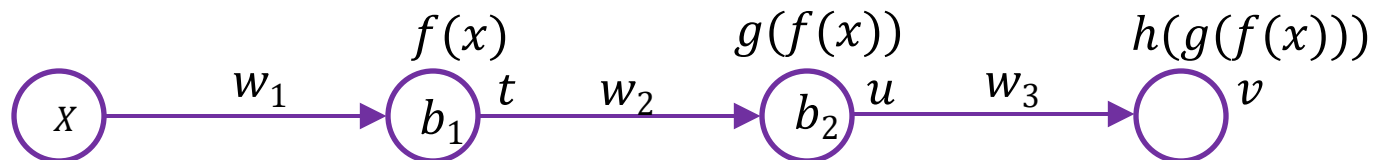
$$v = h(u) = \sigma(u * w_3 + b_3)$$

$$L = L(v) = (v - y)^2$$

v = predicted answer

y = actual answer

- Once we've done a forward pass x and y are fixed values
- To change L the only things we can vary are $w_1, b_1, w_2, b_2, w_3, b_3$





So, we want to compute how a change in w_3 effects L
This is the derivative

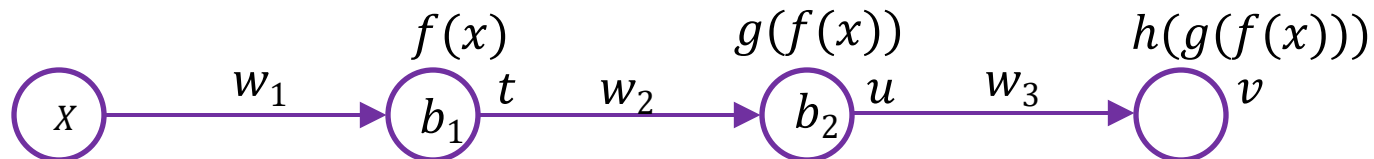
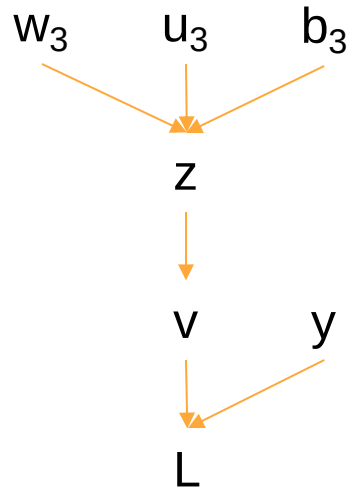
$$\frac{dL}{dw_3} = \frac{dz}{dw_3} \times \frac{dv}{dz} \times \frac{dL}{dv}$$

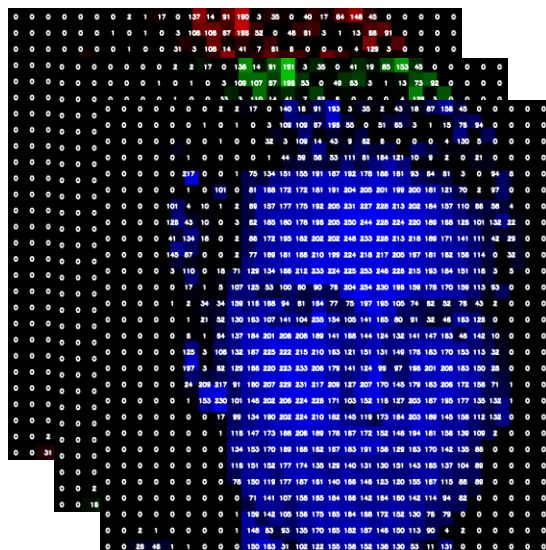
This is the chain Rule

$$\frac{dz}{dw_3} = u, \quad \frac{dv}{dz} = \sigma'(z), \quad \frac{dL}{dv} = 2v - 2y$$

We can use this to
work out how a change
in w_3 impacts L

$$\frac{dL}{dw_3} = u \times \sigma'(z) \times 2(v - y)$$





classification



Human?

We can distinguish the class based on:

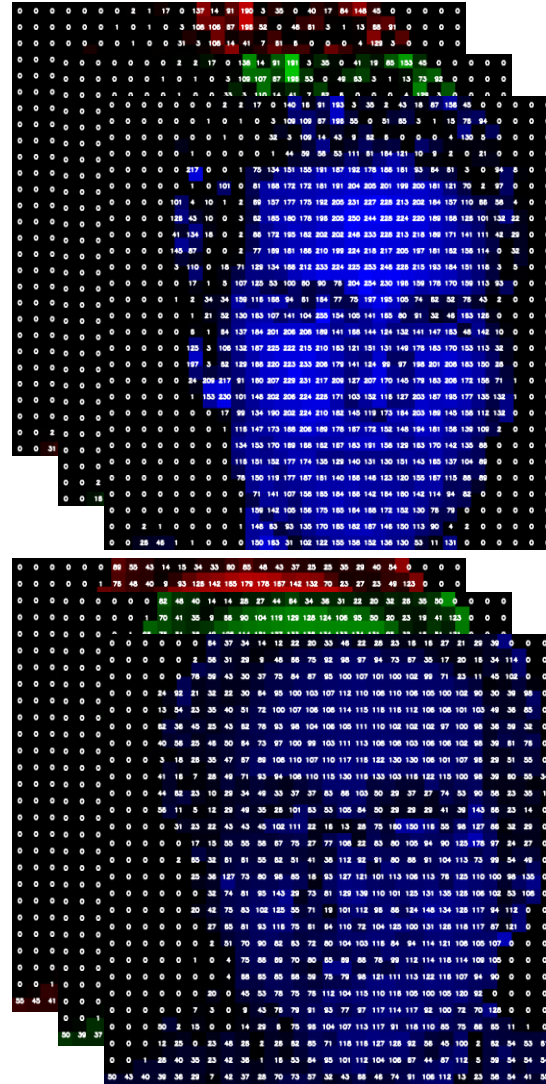
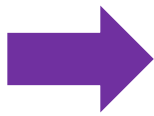
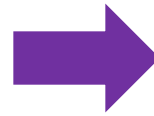
Geometry

Facial Features

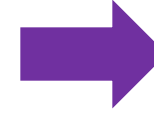
Colours & Textures

Architecture Design

Pattern Recognition / Feature Extraction



classification



Tom?

Looks

Charisma

Facial
Features

classification



Amir?

Physical
Structure



- One could define and extract specific *features* using domain knowledge to classify an image based on its high-level and low-level features.

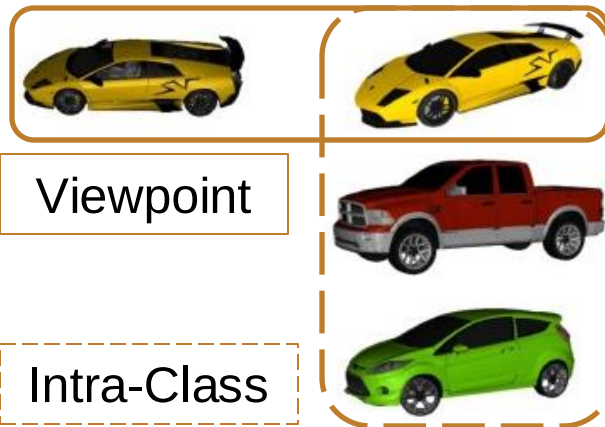


Wings
Beak
Talons



Door
Windows
Roof

- This complex feature detection pipeline is subject to *variations* in images.



Illumination

Scale



Rotation

Occlusion

Deep learning enables hierarchically learning features from images.



Low-Level Features

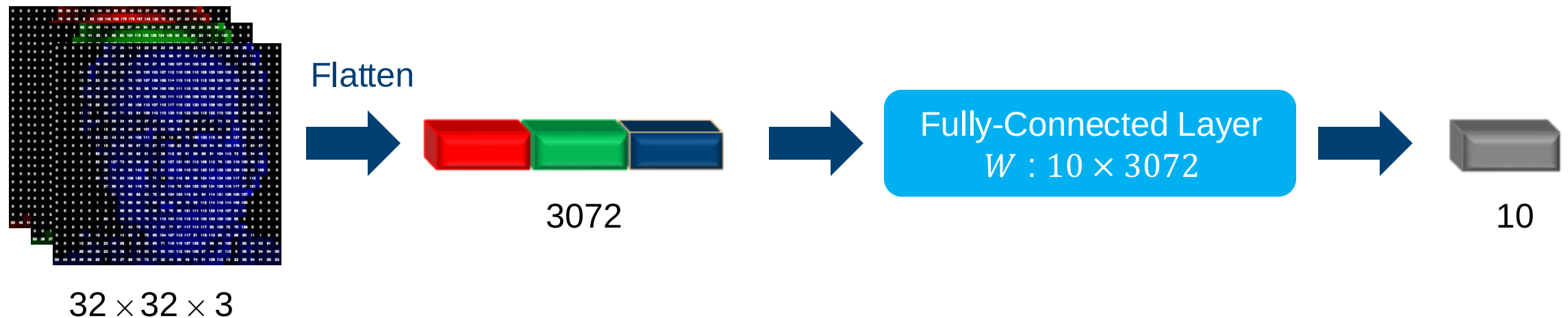
Mid-Level Features

High-Level Features

Convolutions



We have already learned how to use fully-connected layers to classify data - even images by flattening the image.

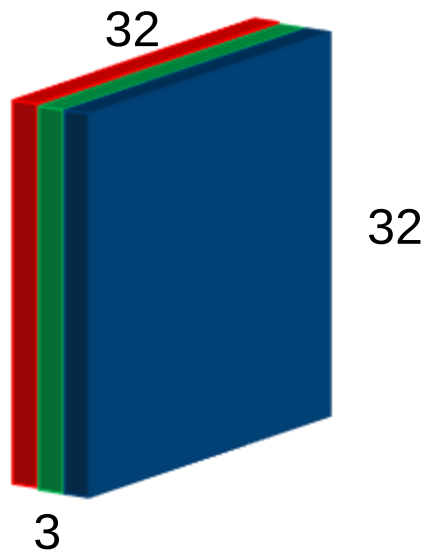


- All spatial structure (meaningful patterns) is lost..!
- Convolutional Neural Networks (CNN) can preserve and learn from the spatial information within the image with fewer parameters.

Convolutions



Instead of connecting every pixel to a neuron, patches are connected to neurons, maintaining spatial structure within the image.



$$7 \times 7 \times 3$$



Weights:
Filter
Kernel

(The feature detector)

The result is a matrix with the number of elements equaling the number of times the filter has moved.

- We will spatially slide the kernel over the image and compute the dot product (*i.e.* convolve the filter with the image).
- The parameters in the kernels are learned.

Convolutions

In Action



For simplicity, let's imagine a $7 \times 7 \times 1$ binary input and a $3 \times 3 \times 1$ kernel.

Input

0	0	1	1	1	0	0
0	1	0	0	0	1	0
1	0	1	0	1	0	1
1	0	0	0	0	0	1
1	0	1	1	1	0	1
0	1	0	0	0	1	0
0	0	1	1	1	0	0

Kernel

1	1	0
0	1	0
1	1	0

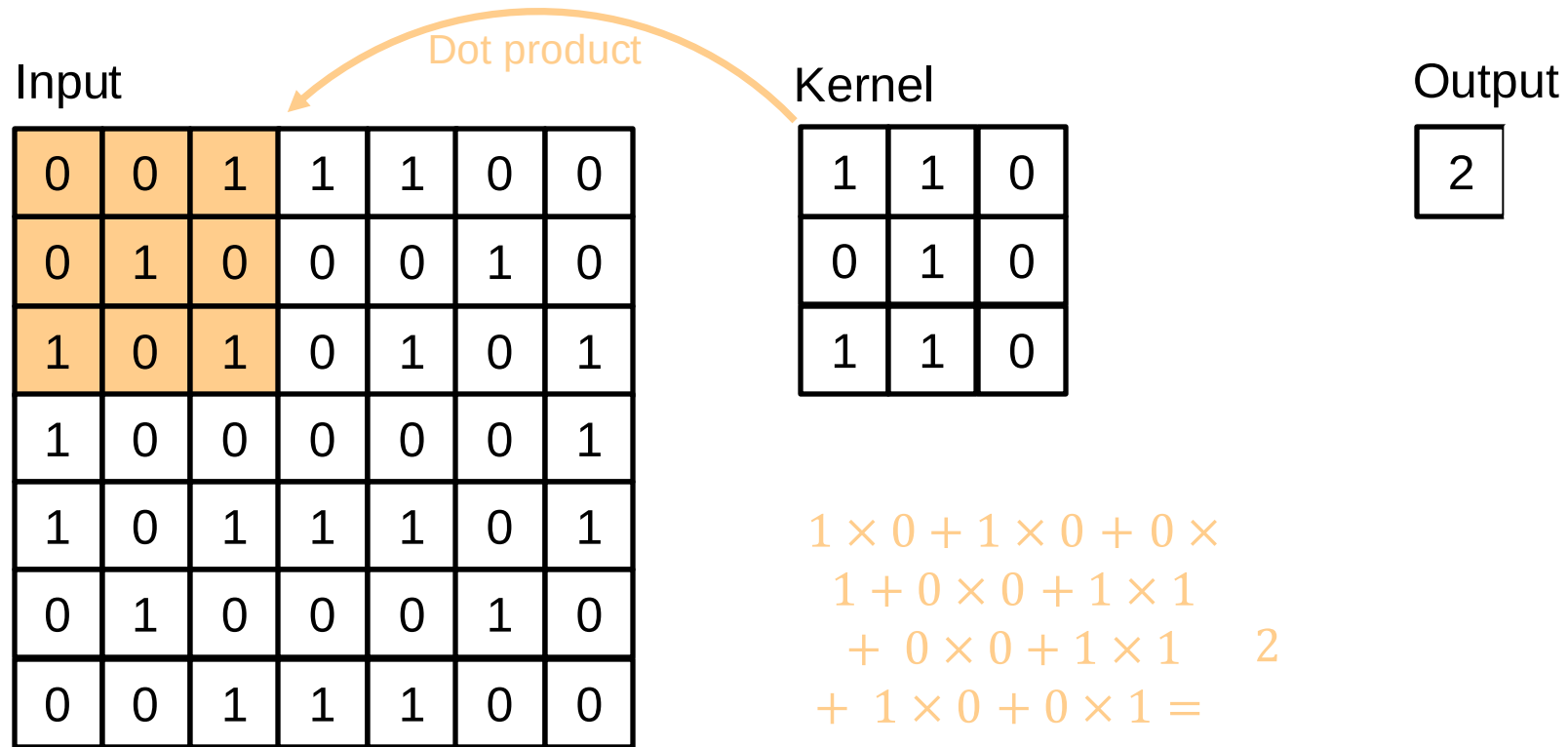
In a CNN, the weights (elements in the kernels) are often randomly initialised and modified as part the backpropagation process.

Note that in the wild, images and kernels are not necessarily binary.

0	0	1	1	1	0	0
0	1	0	0	0	1	0
1	0	1	0	1	0	1
1	0	0	0	0	0	1
1	0	1	1	1	0	1
0	1	0	0	0	1	0
0	0	1	1	1	0	0

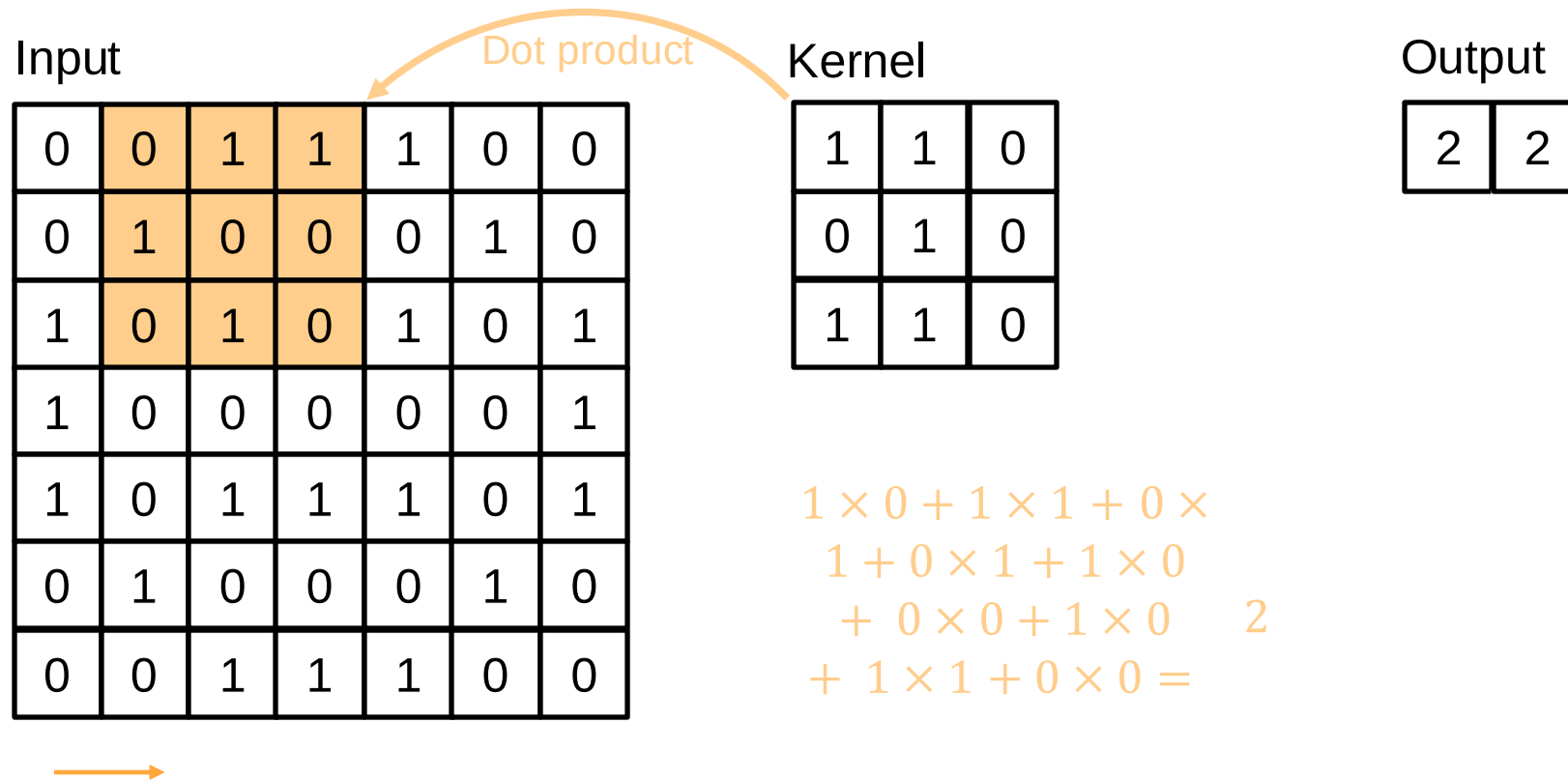


The kernel is placed on the input and the dot product is calculated.





We then slide the kernel over the image by one step (stride = 1).



We slide the kernel to the right one step.
This is convolution with a **stride** of 1.



The kernel is moved one more time and the process is repeated.

Input

0	0	1	1	1	0	0
0	1	0	0	0	1	0
1	0	1	0	1	0	1
1	0	0	0	0	0	1
1	0	1	1	1	0	1
0	1	0	0	0	1	0
0	0	1	1	1	0	0

Dot product

Kernel

1	1	0
0	1	0
1	1	0

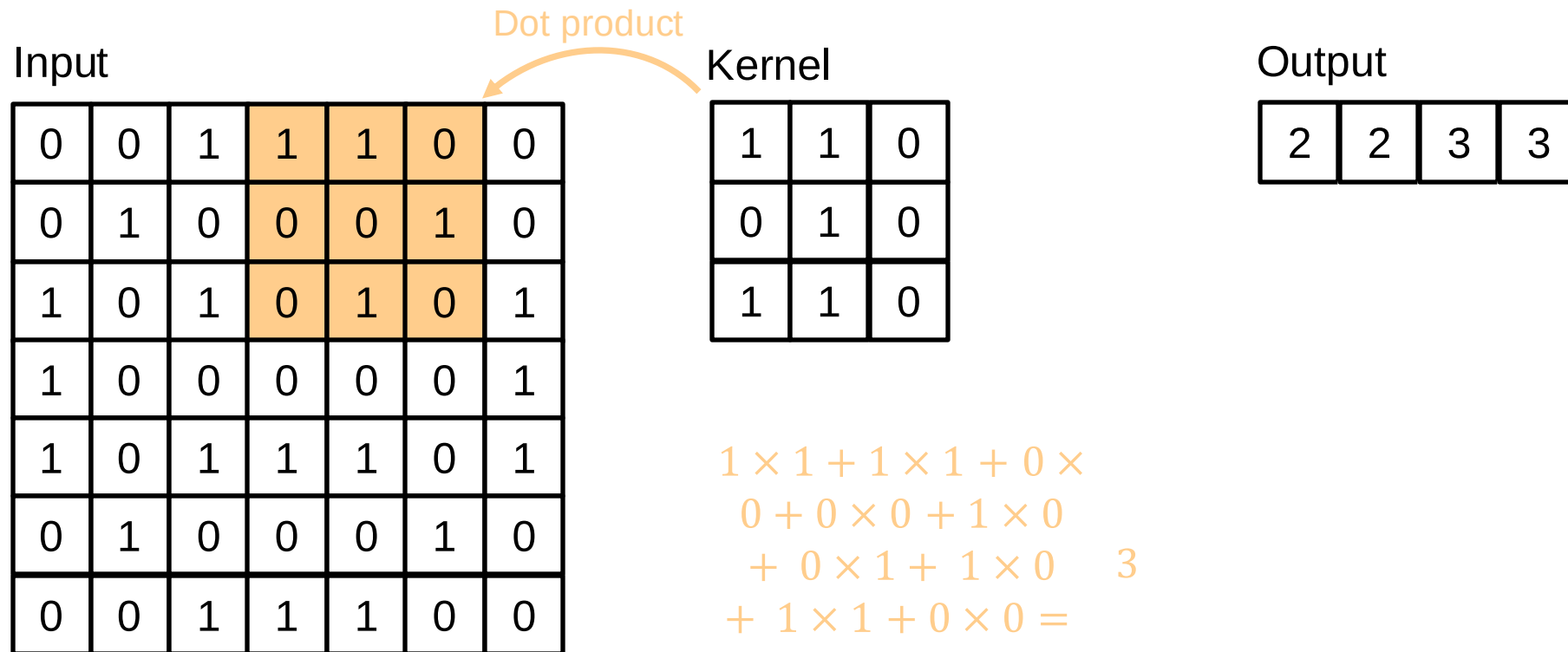
Output

2	2	3
---	---	---

$$\begin{aligned} &1 \times 1 + 1 \times 1 + 0 \times \\ &1 + 0 \times 0 + 1 \times 0 \\ &+ 0 \times 0 + 1 \times 1 \quad 3 \\ &+ 1 \times 0 + 0 \times 1 = \end{aligned}$$

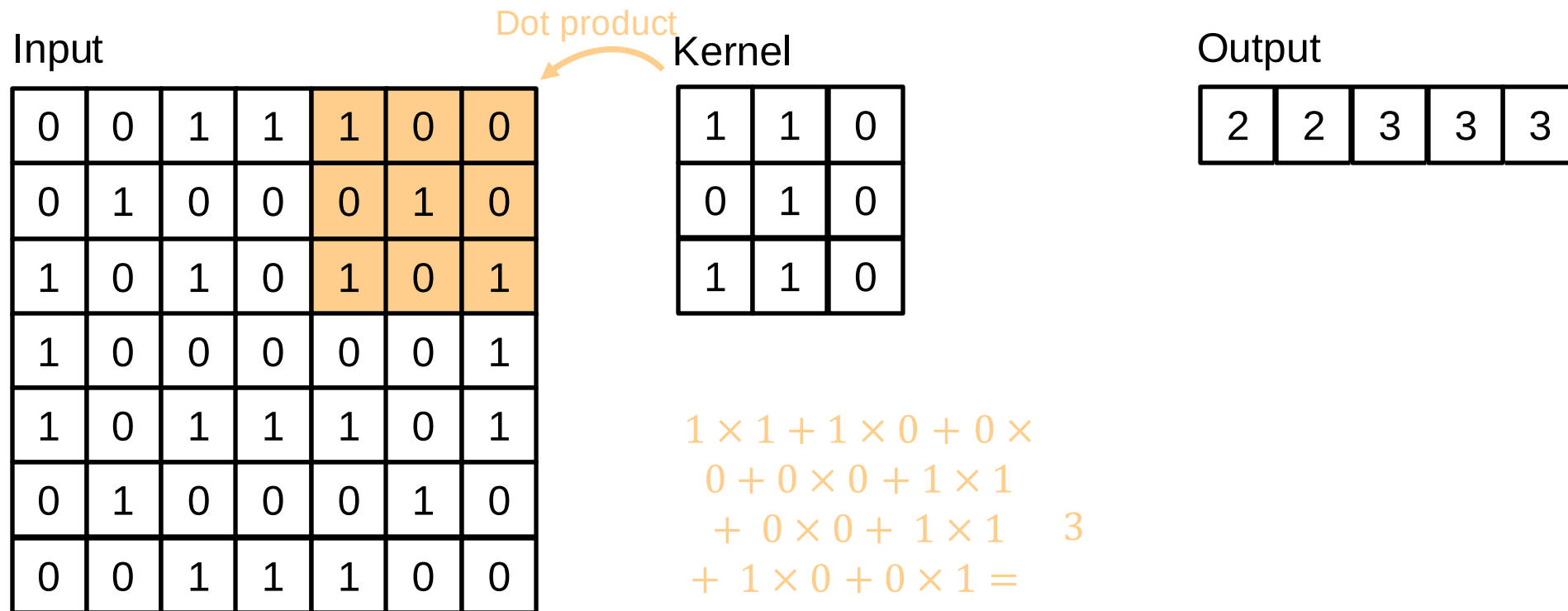


The kernel is moved one more time and the process is repeated.





Note that the size of the output will not necessarily be the same as the input.





The kernel is then moved to the next row of pixels.

Input

0	0	1	1	1	0	0
0	1	0	0	0	1	0
1	0	1	0	1	0	1
1	0	0	0	0	0	1
1	0	1	1	1	0	1
0	1	0	0	0	1	0
0	0	1	1	1	0	0

Kernel

1	1	0
0	1	0
1	1	0

Output

2	2	3	3	3
2				

$$\begin{aligned} &1 \times 0 + 1 \times 1 + 0 \times \\ &0 + 0 \times 1 + 1 \times 0 \\ &+ 0 \times 1 + 1 \times 1 \quad 2 \\ &+ 1 \times 0 + 0 \times 0 = \end{aligned}$$

We will then continue until the entire image is covered..!

Convolutions

In Action



After covering the entire image, an output of 5×5 is created.

Remember:

Image: $32 \times 32 \times 3$

Kernel: $7 \times 7 \times 3$

Note the input and the kernel must have the same depth.

Input

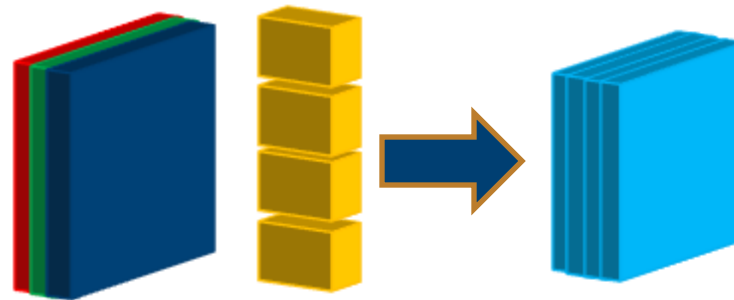
0	0	1	1	1	0	0
0	1	0	0	0	1	0
1	0	1	0	1	0	1
1	0	0	0	0	0	1
1	0	1	1	1	0	1
0	1	0	0	0	1	0
0	0	1	1	1	0	0

Kernel

1	1	0
0	1	0
1	1	0

Output

2	2	3	3	3
2	2	0	1	1
2	2	3	3	2
2	2	1	1	1
2	2	4	4	3



No matter how many channels in the filter and input, the output always has one channel.

By using several kernels, we can get multiple channels of the output (depth).



How do we control the size of the output?

Padding!

0	0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0	0
0	0	1	0	0	0	1	0	0
0	1	0	1	0	1	0	1	0
0	1	0	0	0	0	0	1	0
0	1	0	1	1	1	0	1	0
0	0	1	0	0	0	1	0	0
0	0	0	1	1	1	0	0	0
0	0	0	0	0	0	0	0	0

By adding extra pixels around the input (padding = 1), the output size is 7×7 (same dimensions as input).

There are different types of padding:

- Zero padding
- Constant padding
- Reflection padding
- Replication padding
- ...

Convolutions

Padding



Zero padding:

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	1	2	0	0
0	0	3	4	5	0	0
0	0	6	7	8	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Reflection padding:

8	7	6	7	8	7	6
5	4	3	4	5	4	3
2	1	0	1	2	1	0
5	4	3	4	5	4	3
8	7	6	7	8	7	6
5	4	3	4	5	4	3
2	1	0	1	2	1	0

Constant padding:

c	c	c	c	c	c	c
c	c	c	c	c	c	c
c	c	0	1	2	c	c
c	c	3	4	5	c	c
c	c	6	7	8	c	c
c	c	c	c	c	c	c
c	c	c	c	c	c	c

Replication padding:

0	0	0	1	2	2	2
0	0	0	1	2	2	2
0	0	0	1	2	2	2
3	3	3	4	5	5	5
6	6	6	7	8	8	8
6	6	6	7	8	8	8
6	6	6	7	8	8	8

Convolutions

Output



Size of the output depends on the kernel size, stride and padding.

0	0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0	0
0	0	1	0	0	0	1	0	0
0	1	0	1	0	1	0	1	0
0	1	0	0	0	0	0	1	0
0	1	0	1	1	1	0	1	0
0	0	1	0	0	0	1	0	0
0	0	0	1	1	1	0	0	0
0	0	0	0	0	0	0	0	0

$$O = [(I - K + 2P) \div S] + 1$$

O : size of the output

I : size of the input

K : size of the kernel

P : padding

S : stride

$$O = [(7 - 3 + 2 \times 1) \div 1] + 1 = 7$$

Convolutions

Parameter count

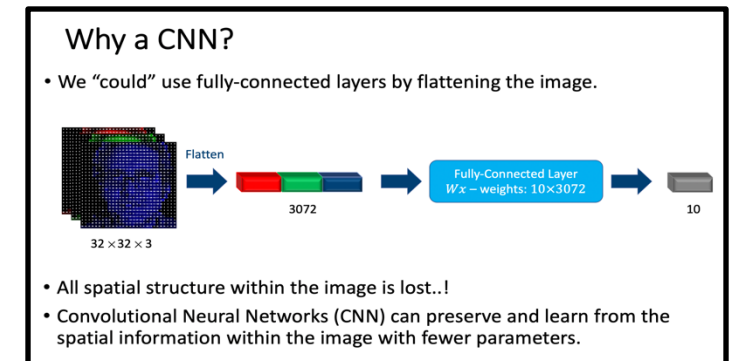
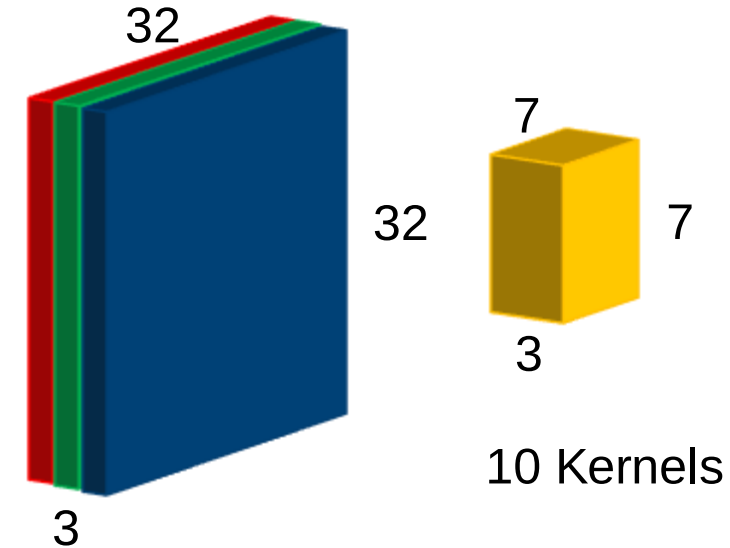


Input size $32 \times 32 \times 3$
10 kernels of size 7×7
stride of 1, padding 3

$$\text{Output size} = \frac{[(32 - 7 + 2 \times 3) \div 1] + 1}{1} = 32$$

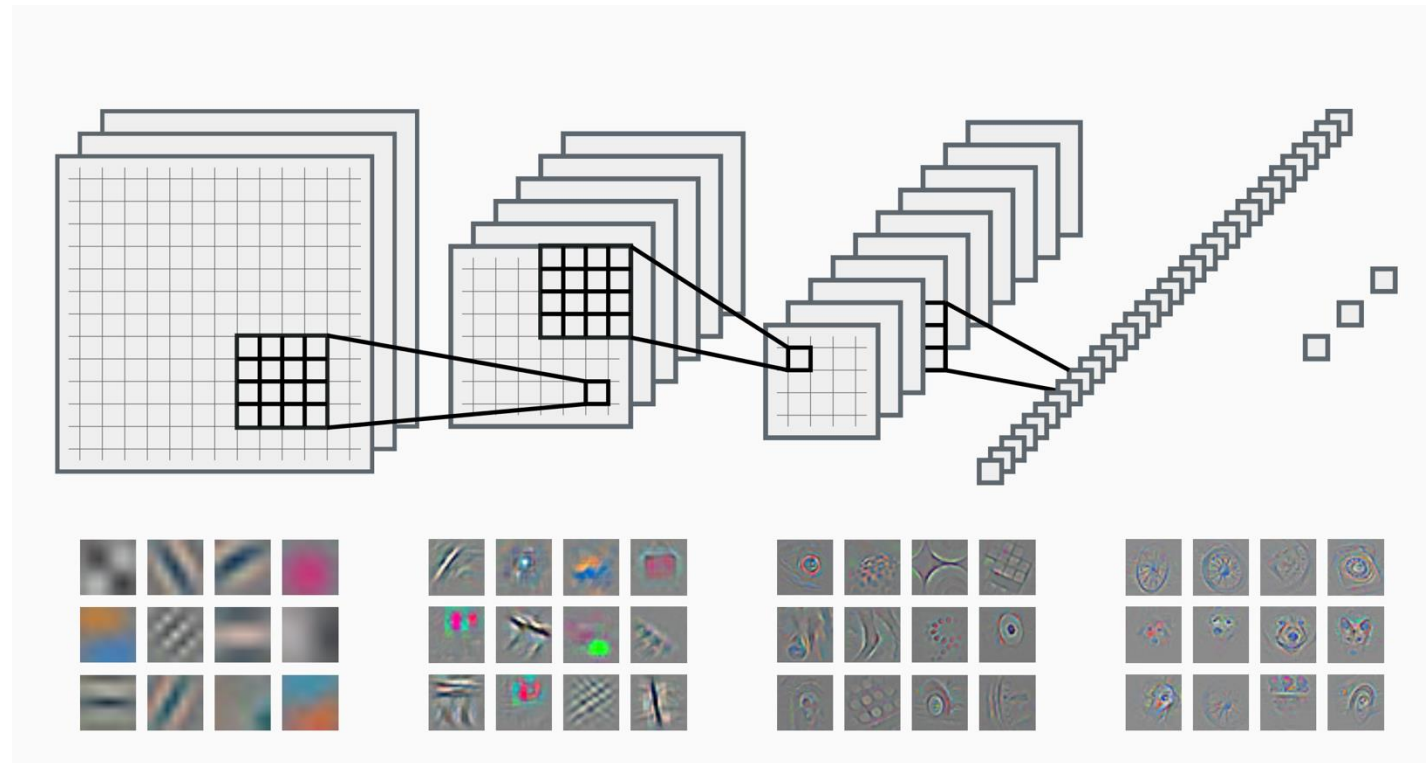
$$\text{Number of parameters} = \underbrace{(7 \times 7 \times 3)}_{\substack{\times 10 \\ \text{parameters} \\ \text{in a kernel}}} + \underbrace{1}_{\text{bias}} \times \underbrace{10}_{\text{number of kernels}} = 1480$$

Remember that in a fully-connected layer, going from the input to a 10-dimensional vector took 30,720 parameters.





- We would generally want to reduce spatial size (resolution) and increase features (channels).
- We typically only use 3×3 , 4×4 and in some cases 1×1 kernels.



Common convolutional patterns

- 4 – 2 – 1: kernel: 4 – stride: 2 – padding: 1 → Halves the spatial dimensions
- 3 – 1 – 1: kernel: 3 – stride: 1 – padding: 1 → Maintains spatial dimensions
- 1 – 1 – 1 → equivalent to a linear layer

Let's Look at Some Code!



Simple Example

Code on GitHub : https://github.com/atapour/dl-pytorch/blob/main/Linear_vs_Conv/Linear_vs_Conv.ipynb

Code on Colab: https://colab.research.google.com/github/atapour/dl-pytorch/blob/main/Linear_vs_Conv/Linear_vs_Conv.ipynb



- To reduce representation size and also introduce translation invariance.

- Average pooling:

1	0	2	3
4	6	6	8
3	1	1	0
1	2	2	4

2×2 Filter with
Stride of 2



2.7	4.7
1.7	1.7

- Max pooling:

Highlights prominent
features and works better
for computer vision tasks.

1	0	2	3
4	6	6	8
3	1	1	0
1	2	2	4

2×2 Filter with
Stride of 2



6	8
3	4

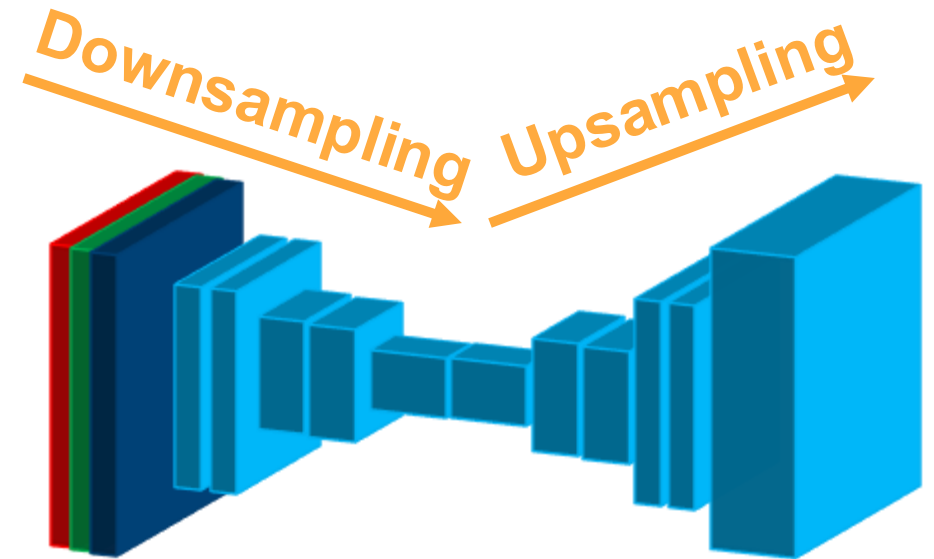
- Global pooling: Instead of down sampling patches of the input feature map, we down-sample the entire feature map to a single value.



Encoders typically reduce the spatial dimensions (compress) but may increase the feature dimensions.
Decoders do the opposite ...

Transpose Convolutions

upsample via learned weights.



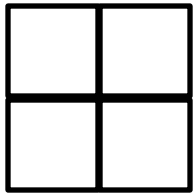
also known as:

- Fractionally-Strided convolution
 - Upconvolution
 - ~~Deconvolution~~
- WRONG

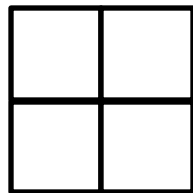


For simplicity, 2×2 transpose convolution with **padding 0**, **stride 1**

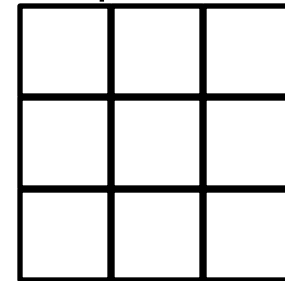
Input: 2×2



Kernel: 2×2



Output: 3×3





2×2 transpose convolution with **padding 0, stride 1**

Input: 2×2

0	1
2	3

Kernel: 2×2

4	5
6	7

The input will behave like a set of weights, multiplying each of its elements by the kernel and placing the results in the output.



2×2 transpose convolution with **padding 0, stride 1**

Input: 2×2

0	1
2	3

Kernel: 2×2

4	5
6	7

step 1

0	0	
0	0	

Step 1



2×2 transpose convolution with **padding 0, stride 1**

Input: 2×2

0	1
2	3

Kernel: 2×2

4	5
6	7

step 1

0	0	
0	0	

step 2

	4	5
	6	7

Step 2



2×2 transpose convolution with **padding 0, stride 1**

Input: 2×2

0	1
2	3

Kernel: 2×2

4	5
6	7

step 1

0	0	
0	0	

step 2

	4	5
	6	7

step 3

8	10	
12	14	

Step 3



2×2 transpose convolution with **padding 0, stride 1**

Input: 2×2

0	1
2	3

Kernel: 2×2

4	5
6	7

step 1

0	0	
0	0	

step 2

	4	5
	6	7

step 3

8	10	
12	14	

step 4

	12	15
	18	21

Step 4



Note that for transpose convolutions, the stride and padding are applied to the output.

2×2 transpose convolution with **padding 0, stride 1**

Input: 2×2

0	1
2	3

Kernel: 2×2

4	5
6	7

step 1

0	0	
0	0	

step 2

	4	5
	6	7

step 3

8	10	
12	14	

step 4

	12	15
	18	21

Putting all the outputs together, **summing** up the elements where they **overlap**.

Final Step

0	4	5
8	28	22
12	32	21

Output: 3×3



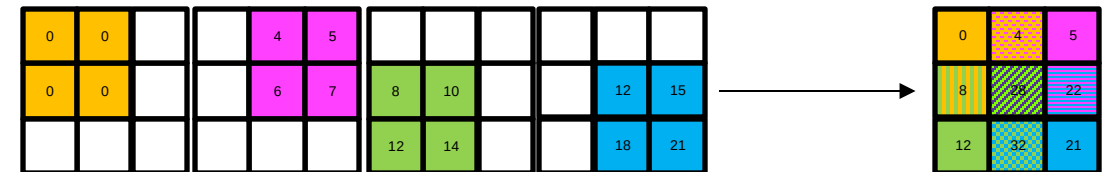
Problem:

Checkerboard Artefacts

Especially a problem when the CNN is generating an actual image.



we summed elements when kernels overlapped in transpose convolutions.



Sum elements where they overlap.

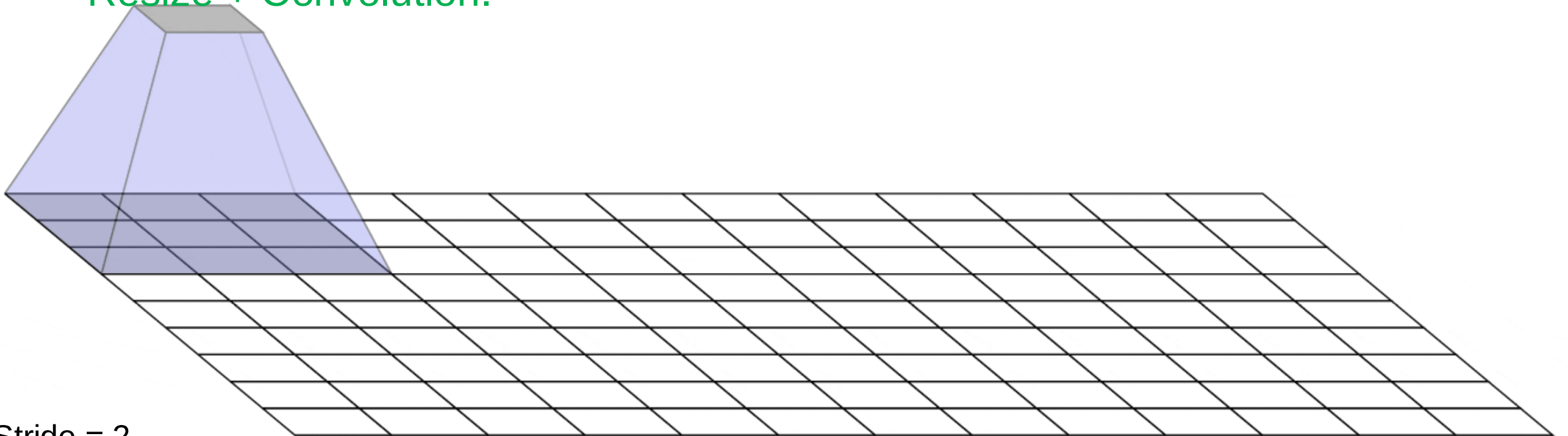


Problem:

Checkerboard Artefacts

Solution:

- Balancing parameters.
- Resize + Convolution.



Stride = 2

Output window size = 3

<https://distill.pub/2016/deconv-checkerboard/>

What we learned today!



Summary

Designing architectures:

- is a scientific process
- design the right shape to transform the data
- choose the right functions to fit the data
- choose your experiments carefully
- what do you know about the signal?
- what does the task need?